

←

Back to Article List

How to build custom OpenClaw skills for your own automations



Ellie Grace Hayes

📅

20/02/2026

OpenClaw skills are almost suspiciously simple. A “skill” is a folder with a `SKILL.md` file – that’s the hard requirement. No SDK / compilation / special runtime. All you need is just YAML frontmatter plus markdown instructions that teach the agent a repeatable way to do something.

That simplicity is the whole point. Tools are capabilities like reading files, running shell commands, browsing the web, calling APIs. Skills are the playbooks that tell the agent how to use those tools in a disciplined way so you get consistent results instead of improvisation.

This article is about writing your own skills. Not installing community ones. Not “here’s a list of skills you should use.” It’s the practical stuff: where to put the folder, how precedence works, what fields in frontmatter matter, how gating works, how to make a skill show up as a slash command and how to keep token overhead from getting silly.

If you want the basics of what skills are and how they behave at runtime, we already have a separate article: [OpenClaw skills guide](#). This one goes deeper on building and distributing your own.

What an OpenClaw skill is

OpenClaw’s docs describe skills as directories containing a `SKILL.md` file with YAML frontmatter plus instructions. That page also explains that OpenClaw loads bundled skills plus local overrides then filters eligibility based on environment, config and binaries on PATH. [OpenClaw skills documentation](#).

Two details matter in real life:

- **Skills do not grant permissions.** If your tool policy blocks `exec` then a skill that relies on shell commands will load but it will fail when it tries to do the work.
- **Skills are operational instructions.** A skill is not a plugin that executes code by itself. It tells the agent what to run and how to format output. That distinction matters for security.

Where skills live and which one wins

OpenClaw loads skills from multiple locations and it uses precedence rules when names collide. The CLI docs for `openclaw skills` summarize that it inspects bundled skills plus workspace and managed overrides and it shows eligibility and missing requirements. [openclaw skills CLI](#).

In practice, think in layers:

- **Workspace skills** in `<workspace>/skills/` for a specific project or agent
- **Managed skills** in `~/.openclaw/skills/` shared across agents on that machine
- **Bundled skills** shipped with OpenClaw

There is also an “extra directories” option in config. The skills config reference documents `skills.load.extraDirs` and it notes these are scanned at the lowest precedence. It also documents the skill watcher settings like `skills.load.watch` and `skills.load.watchDebounceMs`. [OpenClaw skills config](#).

Why you should care about precedence:

- If you copy a bundled skill into your workspace with the same name, your workspace version wins.
- If you ship a custom skill and later install a community skill with the same name, you can accidentally shadow one with the other.
- If two skills have similar descriptions, the model can pick the wrong one even if precedence is correct. Naming and description wording matters.

SKILL.md minimum structure

A skill starts with YAML frontmatter. OpenClaw’s docs are explicit that a skill is a directory containing `SKILL.md` with YAML frontmatter and instructions. [OpenClaw skills documentation](#).

This is the smallest valid example:

```
---

# Weekly report

## Workflow
1. Read last 7 days of logs from the configured path.
2. List merged PRs and closed issues using the configured CLI tools.
3. Produce a markdown report with sections for work done, blockers and next steps.
```

That’s enough for OpenClaw to see the skill and consider it for invocation.

That’s not enough to make it reliable.

Write the body like an operations runbook

Most weak skills fail because the body reads like marketing copy. The agent needs a runbook with deterministic steps, stop conditions and clear output format.

Sections that work well in real skills:

- **What it does** describing when the skill should be used
- **Inputs needed** stating required paths, URLs, IDs and credentials
- **Workflow** with numbered steps and exact commands
- **Output format** so results look the same every run
- **Guardrails** for safety and hallucination control
- **Failure handling** explaining what to do when a command fails or data is missing
- **Examples** showing typical queries and expected output shape

Notice what is missing: a long intro. Skills should feel like checklists you’d hand to a tired on-call engineer at 3am.

Good frontmatter fields

The official skills docs cover the basics of how skills are discovered and loaded. For configuration fields beyond name and description, a useful mental model is:

- **User experience fields** controlling how a skill appears and how you call it
- **Invocation control fields** controlling if the model can invoke it
- **Metadata fields** for gating and installer specs

Name and description

These are the most important fields because they decide discoverability and correct matching. Keep them short and specific. If your description overlaps with another skill, the model will sometimes pick the wrong one even if the workflow is perfect.

user-invocable and slash commands

OpenClaw supports user slash commands for skills when the skill is user-invocable. That’s the “I want to run this explicitly” path. If you want a skill that never triggers automatically and only runs via a slash command, you combine `user-invocable` with a separate invocation control field that removes it from model context.

Claude’s own skill docs say it plainly: “The `user-invocable` field only controls menu visibility, not Skill tool access.”

That sentence is about Claude Code, but the point transfers cleanly. Don’t confuse “visible to user” with “available to the model.”

disable-model-invocation for skills you only run manually

If a skill is dangerous or just too broad, remove it from model invocation and keep it as a manual tool. Claude’s docs recommend `disable-model-invocation: true` to remove a skill “from context entirely.”

That makes a big difference for safety and for token overhead because the skill name and description are not injected into every turn.

command-dispatch tool for deterministic commands

There’s a neat pattern for skills that should behave like deterministic commands: bypass the model entirely and dispatch directly to a tool. A recent community thread describes this as “command-dispatch: tool” and notes it can be used when you want a skill command to be deterministic.

This is useful for things like:

- calling a tool with raw args and no LLM interpretation

It also reduces the chance of the model “helpfully” doing extra steps you did not ask for.

metadata openclaw object

OpenClaw supports a `metadata` field in frontmatter that is a single-line JSON object. Inside it, `openclaw` can define gating rules, platform restrictions and installer specs. The key formatting constraint is that the metadata must stay single-line JSON so the parser can read it. This is discussed in multiple community references and it is a common source of “why does my skill not load.”

A practical example:

```
---
name: server-health
description: Check CPU, memory, disk and network then report warnings.
metadata: {"openclaw":{"emoji":"????", "requires":{"bins":["df", "ps", "uptime"], "os":["linux"]}}}
---
```

Gating and eligibility checks

Gating is what decides if a skill becomes eligible in the current environment. OpenClaw’s skills docs mention that it filters skills at load time based on environment, config and binary presence. [OpenClaw skills documentation](#).

In practice, gating answers a simple question: “Should this skill be usable right now?”

Common gating types

- OS gating for linux, darwin, win32
- Binary gating via `requires.bins` so the agent does not try commands that are missing
- Environment gating via `requires.env` so the skill is not eligible without credentials
- Config gating via `requires.config` when you need a config flag enabled

When a skill is not eligible, you want visibility. The `openclaw skills` CLI exists for this exact reason, and the docs mention it helps you see what’s eligible versus missing requirements. [openclaw skills CLI](#).

Debugging eligibility

These are the commands I actually use:

```
openclaw skills list
openclaw skills list --eligible
openclaw skills list --verbose
openclaw skills info my-skill
openclaw skills check
```

The point is to stop guessing. If your skill is “installed” but not eligible, it’s almost always missing a binary or env var, or it is blocked by OS gating.

Skill directories and the watcher

You don’t want to restart the gateway every time you tweak a workflow step. OpenClaw supports watching skill folders and refreshing the snapshot when `SKILL.md` changes. The config docs show `skills.load.watch` and `skills.load.watchDebounceMs` defaults. [OpenClaw skills config](#).

That means you can iterate like this:

- run the skill
- see where it went wrong
- edit SKILL.md
- run it again on the next turn

If you do not see changes taking effect, start a new session. Some environments cache the skills snapshot at session start.

A skill template for production

This template is the one I recommend copying for your first few skills. It includes the practical sections and the guardrails that prevent “helpful chaos.”

```
metadata: {"openclaw":{"emoji":"????","requires":{"bins":["bash","date"],"os":["linux","darwin"]}}}
---

# Weekly ops report

## What it does
Builds a weekly report from local logs and optional issue tracker CLI output.

## Inputs needed
- LOG_DIR: directory with daily log files
- OPTIONAL: a CLI tool for your issue tracker if installed (gh, jira, etc.)

## Workflow
1) Verify LOG_DIR exists. If not, ask the user for the correct path.
2) Read logs from the last 7 days.
3) Extract:
  - incidents
  - recurring warnings
  - changes deployed
4) If gh is installed, list merged PRs and closed issues for the last 7 days.
5) Produce the report in the output format below.

## Output format
Return markdown with headings:
- Summary
- Incidents and fixes
- Changes deployed
- Customer-impacting items
- Next week

## Guardrails
- Never fabricate PR numbers, issue IDs or incident counts.
- If data is missing, state exactly what is missing.
- Do not restart services or change config.
- Stop after scanning 7 days of logs.

## Failure handling
- If a command fails, include the command and the error text then stop.
- If parsing is ambiguous, show the raw lines you used.

## Examples
User: "Generate the weekly ops report for /var/log/myapp"
Assistant: returns the report in the specified format.
```

Two reasons this works:

- it forces input validation early
- it prevents the agent from inventing data when inputs are missing

Skill design patterns (that save you time later)

Keep scope narrow

If your skill does “everything about deployments” it will overlap with other skills and it will misfire. Keep scope specific. “Generate a weekly report from these logs” is good. “Manage my operations” is a disaster.

Prefer read-only first then add writes later

Most automations can start as read-only reporting. Once you trust the logic you can add actions like creating a report file or moving items into an archive folder.

Make destructive actions opt-in

If a skill can delete files or change production settings, make it require explicit confirmation inside the runbook. Also consider keeping it out of model invocation using `disable-model-invocation: true` so it only runs when you call it directly.

Put constants into config not into the body

Hardcoding “/var/log/app” into SKILL.md is fine for a private skill. If you want to share it, move paths into documented variables and put defaults in your config so users can override without editing the skill itself.

Token overhead and how to keep it under control

Every eligible skill adds text to the system prompt. Even when the skill is not used, the model sees a summary list of skills, and that list costs tokens. OpenClaw’s skills docs mention that it loads bundled and optional overrides then injects summaries.

What I do to keep this sane:

- keep descriptions short and specific
- disable skills you are not using right now
- use `disable-model-invocation` for skills you only call manually

Three ways to create a skill

Create a skill from chat then edit it

This is the fastest for a first draft. You describe the automation, OpenClaw generates `SKILL.md`, then you edit it into a runbook. Do not trust the first draft. Treat it like scaffolding.

Write SKILL.md manually

Create a folder then add SKILL.md:

```
mkdir -p ~/.openclaw/skills/my-skill
nano ~/.openclaw/skills/my-skill/SKILL.md
```

Then verify detection:

```
openclaw skills list --eligible
openclaw skills info my-skill
```

The CLI docs cover these commands. [openclaw skills CLI](#).

Fork an existing skill then narrow it

This is underrated. If someone already built 80 percent of what you need, copy it and make it smaller. The most common improvement is guardrails. Community skills often lack stop conditions and safety rules.

Publishing and sharing your skill

You have a few distribution paths and each has tradeoffs.

Share the folder in Git

Private team skills often live in a private repo. People clone into `~/.openclaw/skills` or into a workspace skills directory. This is simple and auditable.

Publish to ClawHub

ClawHub is OpenClaw's public skill registry. The project's own repo describes ClawHub as a public registry for text-based agent skills, designed for browsing and a CLI-friendly API.

If you publish publicly, you need to treat your skill like open source. That means: clear README, no secrets, explicit requires list and safe defaults.

Package skills for install via npm

Some teams package skills inside an npm package for internal distribution, and OpenClaw can install that package. This is convenient when your org already does npm distribution.

For public distribution, ClawHub tends to be the more discoverable channel.

Security warnings you should not ignore

Skills are instructions that can tell an agent to run commands. That means skill ecosystems attract attackers, and in early 2026 that became very visible.

The Verge reported on malicious OpenClaw skills distributed through ClawHub, including skills that attempted to trick users or agents into running commands that install infostealers.

Tom's Hardware also covered malicious skills uploaded to ClawHub that targeted crypto users and relied on social engineering and terminal command execution.

What this means for you as a skill author:

- Never instruct users to run obfuscated shell commands in your SKILL.md.
- Never require users to paste secrets into chat. Use env var gating and config injection.
- Keep binaries and network calls minimal. Declare only what you need.
- Write guardrails that prevent the agent from doing extra actions outside the task.

What this means for you as a skill user:

- Read every SKILL.md you install. Treat it like code review.

- Keep exec approvals on until you trust the behavior.

If you want a more complete hardening guide, use our tutorial: [OpenClaw security best practices guide](#).

Troubleshooting custom skills

Skill not listed at all

- confirm the folder is in a scanned skills directory
- confirm the filename is exactly `SKILL.md`
- confirm YAML frontmatter starts and ends with `---`

Then run:

```
openclaw skills list
openclaw skills check
```

The CLI docs cover these commands. [openclaw skills CLI](#).

Skill is listed but never triggers

This is almost always description overlap or vague wording. Tighten the description so it matches a clear intent. “Monitor disk usage and alert” triggers better than “system helper.”

If it is a skill you only want to run explicitly, remove it from model invocation and use a slash command.

Skill triggers but fails on command execution

Check tool permissions first. Skills do not grant permissions. If your tool policy blocks exec, the workflow cannot run commands.

Then check gating. If your skill expects `jq` and it is not installed, either install it or remove it from the workflow. Use `openclaw skills list --verbose` to see missing requirements.

Metadata parse errors

If you used metadata, keep it as single-line JSON. Multi-line metadata is a common source of parser failures in third-party guides.

Edits do not take effect

Confirm skill watching is enabled. The skills config docs show `skills.load.watch` and the debounce setting. [OpenClaw skills config](#). If it still does not update, start a new agent session.

Your idea deserves better hosting

 24/7 support  30-day money-back guarantee  Cancel anytime

Starter

1-Year



Semi-Annually

Monthly

1 GB RAM VPS

~~\$2.99~~

\$1.99 /Monthly

FAQ

How do I create a custom OpenClaw skill?



Create a folder in `~/.openclaw/skills/undefinedlt;skill-nameundefinedgt;/` or in your workspace `skills/` directory then add a `SKILL.md` with YAML frontmatter containing `name` and `description`. Write the body as a step-by-step workflow with guardrails and output format. OpenClaw's skills docs describe this directory plus SKILL.md structure. docs.openclaw.ai/tools/skills.

How do I debug why my skill is not eligible?



How do I make a skill only run via slash command?



How do I make a deterministic slash command that bypasses the model?



How do I publish my OpenClaw skill?



Are OpenClaw skills safe to install from the internet?



Automate faster, for less

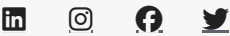
Bring your winning ideas to life with AMD power, NVMe speed and unmetered bandwidth. Everything backed by 24/7 support, plus a 30-day refund period.

Launch your OpenClaw VPS



LumaDock offers fast, affordable VPS hosting for developers, businesses, and gamers, including GPU and Game Server Hosting, powered by NVMe storage and AMD EPYC.

LumaDock is a LifeinCloud LTD brand. London W1S 1HN, United Kingdom, registered in England and Wales, company No. 15844051



- Products

▼
- App hosting solutions

▼
- Features

▼
- Resources

▼
- Solutions by use case

▼
- Get help

▼
- Company

▼

[Privacy policy](#)

[Terms of service](#)



Copyright © 2026 LumaDock. All Rights Reserved.